



P17

Least-Privilege Orchestration: Scope Tightening and Re-authorization in the Transfer Bundle

最小权限编排：Transfer Bundle 的 scope 收紧

Least-Privilege Orchestration: Scope Tightening and Re-authorization in the Transfer Bundle

Working Title (EN): Least-Privilege Orchestration: Adding a Scope Field to the Transfer Bundle so Owners Can Only Touch the Stages They Are Authorized For

Shard D • Market and Architecture paper P17 • structural upgrade of v7.5.0 P10 "Agent-as-Tool and TransferBundle" • cross-referenced with P2 "BFT-lite stopping decisions," P3 "fault governance," P11 "internal task market" • UDOS Reasoning Engine v7.6.0 • Fang Wenxin • 2026-09-21

One sentence first: the real management bottleneck in multi-agent orchestration is not that the Leader "cannot understand" what sub-agents are doing, but that the Leader hands **excessive privileges** to sub-agents in one shot; v7.5.0's TransferBundle has only one `owner` (who is responsible), v7.6.0 adds a `scope` (which stages this owner **is allowed to operate**), letting privileges automatically tighten as progress advances and be re-granted on handoff—this is the first time the old engineering principle of "least privilege" is written into UDOS's handoff contract.

Evidence caliber (stated only once in the whole paper): this paper is a **design proposal**, containing no new UDOS measured numbers; scope field benefits all marked "design proposal • to implement," using [RESULT NEEDED] placeholders, not written as measured. The only external dialogue object is ClawArena-Team (arXiv:2606.31174, existence verified online 2026-09-21); its "management bottleneck in privilege granting," "cost and management quality approximately decoupled," "pure execution-style scoring, no LLM judge" are paper directions, while precise thresholds and formula strings like "no LLM exceeds 50% workspace privilege precision," "cost span over a hundredfold while total score span under fourfold," "SMS = task correctness × least-privilege factor × modality routing factor" were not verbatim hit in abstract-level verification, this paper all relaying as "paper report • not independently reviewed," not hardcoding concrete percentages. UDOS old numbers only point back to v7.5.0 papers (P10 six-field contract, 26 tests verified).

Abstract

When a Leader Agent simultaneously manages several sub-agents, the engineering failure point most easily overlooked is not "whether the task can be done" but "how much it is allowed to do." ClawArena-Team (arXiv:2606.31174) reports on a multi-turn, multimodal, multi-directory sub-agent orchestration benchmark: the Leader's management bottleneck concentrates in the **privilege granting** link, and API cost is approximately decoupled from management quality—spending more money for a stronger Leader model does not proportionally buy better orchestration quality. These two experiences point to the same structural judgment: **management is a dimension independent of model capability, and its first handle is the privilege boundary, not perception or reasoning.**

v7.5.0's TransferBundle fixes the handoff package to six fields (Goal/Context/Done/Trace/Owner), where `owner` only answers "who is responsible for this leg now," not "what it is allowed to touch." This paper proposes the v7.6.0 least-privilege upgrade: beside `owner`, add a new `scope` field, explicitly recording "the set of stages this owner is currently allowed to operate." `advance()` removes completed stages from scope (**automatic tightening**), `handoff()` when handing control to the next leg has the new owner re-apply for and validate a smaller scope (**re-authorization**). Any write exceeding scope is mechanically rejected at the `validate()` stage, returning the typed error `scope_violation`, rather than

silently passing. This paper gives the field's contract definition, three invariants (scope monotonically tightens, out-of-bounds rejected, re-authorization must pass validate), the compatibility path with P10's existing six fields/26 tests, and leaves "whether least privilege really reduces boundary crossing and mis-isolation" to a preregistered factorial experiment. This paper honestly states: scope is a **design proposal** • **to implement**, its cost-reduction/error-reduction benefits currently have no UDOS measured data, wherever concrete numbers are involved all [RESULT NEEDED].

Keywords: multi-agent orchestration; least privilege; TransferBundle; privilege tightening; handoff contract; fault governance; management bottleneck; design proposal

Structured Abstract (background problem → method/argument → evidence → contribution)

- **Background problem:** multi-agent orchestration long attributed "management difficulty" to "models not smart enough," so the default solution is "change to a stronger, more expensive Leader." But ClawArena-Team's execution-style scoring suggests: the management bottleneck actually falls in **privilege granting**, and cost is approximately decoupled from management quality —changing to expensive models is not an effective lever for orchestration quality.
 - **Method/argument:** writing "least privilege" (the old security-engineering principle of "giving only the minimum privileges needed to complete the current task") from a slogan into a new TransferBundle field `scope`; privileges automatically tighten with `advance`, re-granted with `handoff`, out-of-bounds mechanically rejected by `validate`.
 - **Evidence clues:** (1) ClawArena-Team's paper direction (privilege bottleneck, cost-quality decoupling, no-LLM-judge execution-style scoring, paper report • not independently reviewed); (2) v7.5.0 P10's verified six-field contract and 26 tests (scope is an increment on it, not overturning existing invariants); (3) security-engineering least-privilege principle (textbook consensus, this paper not claiming to originate it).
 - **Contribution:** (1) upgrading "owner = who is responsible" to "owner+scope = who is responsible, which stages can be touched"; (2) giving three contract invariants that can be pinned by tests; (3) providing a v7.5.0-compatible migration path and preregistered factorial experiment design, honestly marking all benefits as [RESULT NEEDED].
-

1 Introduction: Management Bottleneck Is Not "Whether Understandable," but "How Much Privilege Given"

When multi-agent systems move from "a few Agents collaborating" to "one Leader managing a group of sub-agents," a common intuition is: poor management because the Leader is not strong enough; the solution is changing the Leader to a stronger, more expensive model. This intuition treats "management" as a byproduct of "reasoning capability."

ClawArena-Team (arXiv:2606.31174) gives a counter-direction signal worth taking seriously on the sub-agent orchestration benchmark. It measures the Leader's management capability with **pure execution-**

style scoring (i.e. scoring by actual task output, **not introducing LLM-as-Judge**, avoiding the judge model's own bias), and reports two points (paper report • not independently reviewed):

1. Management bottleneck concentrates in **privilege granting**—the Leader is not "unable to understand what sub-agents are doing" but "gave sub-agents excessive workspace privileges," so unauthorized operations occur at high frequency;
2. API cost is **approximately decoupled** from management quality—the cost span is large, while the management total score span is far smaller than the cost span, i.e. the equation "expensive model = good management" does not hold.

Together these point to a structural judgment: **management is a dimension independent of model capability, its first handle being "privilege boundary," not "perception breadth" or "reasoning depth"**. If this judgment holds, the correct engineering action is not "making the Leader bigger" but "cutting privileges smaller, more accurately, tightening with the flow."

This paper does precisely this. v7.5.0's TransferBundle (P10) already wrote handoff as a mechanically verifiable six-field contract, but its `owner` field only answers "who this leg belongs to," not "which stages it is allowed to touch." This paper adds a `scope` on it, writing the old security-engineering principle of "least privilege" into the handoff contract for the first time.

Three things must be clarified first: first, this paper does not re-prove the least-privilege principle (it is security-engineering textbook consensus), this paper only answers "what it looks like in UDOS's handoff contract"; second, this paper **does not claim** scope has been measured to reduce cost/error—it is a design proposal; third, this paper does not take ClawArena's precise thresholds as its own experimental data, only borrowing its direction as design motivation.

2 From owner to owner+scope: A Field-Level Structural Upgrade

2.1 What v7.5.0's owner Lacks

Reviewing P10: a handoff has the sender construct a `TransferBundle`, six fields being `goal` (objective), `context` (fact dictionary), `done` (completed stage list), `todo` (to-do stage list), `trace` (event sequence number list), `owner` (current responsible party). `handoff(next_owner)` first calls `validate()`, if problems it does **not transfer** responsibility, only if no problems changing `owner` to `next_owner`.

This contract is solid, but it implicitly assumes: **as long as owner is the "right person," it can act on everything in todo**. `owner` is a **single-point responsibility pointer**, it guarantees "someone is responsible" but does not guarantee "this person can only be responsible for the cell they should touch." In other words, P10 solved "no one closes" but did not solve "privilege boundary crossing"—an Agent wrongly pushed into the `owner` position, or an honest Agent injected with wrong instructions, under owner mode can still perform writes on all to-do stages.

This is precisely the failure surface ClawArena diagnoses: **responsibility boundary ≠ privilege boundary**.

2.2 The New scope Field

This paper proposes upgrading six fields to seven, beside `owner` adding:

- `scope`: the set of stages the current owner **is allowed to operate**, a proper subset of `todo` (empty set allowed, empty meaning "read-only observation, no writes").

A more intuitive writing: v7.5.0's owner answers "who this leg belongs to," v7.6.0's owner+scope answers "who this leg belongs to, and which cells it is currently allowed to touch."

`scope` is not static configuration but **dynamically changes with task progress**:

- After `advance(bundle, stage, ...)` succeeds, removing `stage` from `scope`—once the task is done, immediately revoking write privilege for that stage. This is **automatic tightening** (`scope` shrinks monotonically as stages advance).
- At `handoff(next_owner)`, the new owner does not "inherit all `todo` privileges" but **re-applies** for a `new_scope` it declares needed; only when `new_scope ⊆ todo` and passing `validate()` do control and the new scope transfer together. This is **re-authorization on handoff**.
- Any write exceeding the current `scope` (e.g. an owner trying to operate a stage not in its scope) is rejected at the `validate()` stage, returning the typed error `scope_violation`, and recording the violating stage, current scope, requester into `trace`. It **does not silently pass** nor **silently fail**—the same spirit as P10 rejecting silent forwarding.

2.3 Three Contract Invariants

`scope`'s value must land on testable invariants, not "feels safer." This paper proposes three:

- **I1 (scope monotonically tightens)**: within the same owner term, `scope` can only shrink or stay with `advance`, cannot be quietly enlarged. Formally, after two adjacent `advance` s $scope_{\{t+1\}} \subseteq scope_t$.
- **I2 (out-of-bounds rejected)**: any write to $s \notin scope$ returns `scope_violation`, producing no side effects, not changing `done / todo / owner`.
- **I3 (re-authorization must pass validate)**: the new `scope` a `handoff` transfers must satisfy `new_scope ⊆ todo` and be non-empty (unless an explicit read-only handoff), otherwise not transferring responsibility, fully isomorphic with P10's "validate failure does not change owner."

These three are parallel to P10's existing "handoff failure does not change owner," "`done ∩ todo` rejected," "missing owner → no_closer," not replacing. They should all be pinned by new unit tests, evidence grade target `verified` (same standard as P10's 26 tests).

3 Why "Privilege" and Not "Model Size"

Figure 1 Separation of privilege boundary and responsibility boundary

Figure 1 Left: v7.5.0's owner is a "single-point responsibility pointer," who is responsible = who can act on all to-do; right: v7.6.0's owner+scope decouples responsibility and privilege—who is responsible, and which cells it is currently allowed to touch, recorded separately. Conceptual illustration, not measured data.

3.1 What Cost-Quality Decoupling Means

ClawArena-Team's execution-style scoring report (paper report • not independently reviewed) gives a hint important for engineering scheduling: in its scenario, the Leader model's cost span is large, but the management quality total score span is far smaller. This is not saying "model size is completely useless" but **"changing to a larger Leader model" is not a cost-effective lever for orchestration quality**—if your real bottleneck is privilege crossing, then changing the Leader from a cheap model to a flagship, it will still open sub-agents' workspaces too large, because the problem is not whether it "can reason" but how large a hand the system "gave it."

In P0b's words, this is again a confirmation of "**structural prior determines the upper limit**": a structural choice like the owner field, before model size, determines on which failure class the system will overturn.

3.2 Execution-Style Scoring: Not Giving Judging to Another LLM

Another engineering choice of ClawArena worth borrowing is **pure execution-style scoring, no LLM judge**. In multi-agent evaluation, "letting one LLM be judge scoring other LLMs" (LLM-as-Judge) is common, but the judge model itself has bias and stance, introducing new noise into evaluation. Execution-style scoring judges by task **actual output**, changing the judge from "model self-assessment" to "whether output is achieved."

This agrees with UDOS's consistent "using mechanical contracts and rerunnable tests instead of model self-assessment" (P10's `validate()`, P12's evidence grading both this orientation). This paper's acceptance of scope likewise does not go "letting the Leader self-prove proper management" but "whether `scope_violation` is mechanically triggered, whether crossing is really rejected" type criteria that can be pinned by test assertions.

4 scope's Dynamic Lifecycle: Tightening and Re-authorization

Figure 2 scope tightens with advance, re-granted with handoff

Figure 2 The scope trajectory of one task progress: on handoff the new owner applies for a small scope (re-authorization), each completed stage removed from scope (automatic tightening), out-of-bounds writes rejected at validate and recorded in trace. Conceptual illustration, not measured data.

Drawing the scope trajectory of one task from dispatch to closing (Figure 2), one sees it oscillates between two actions:

1. **Re-authorization (handoff)**: when control passes to the next leg, the new owner declares `new_scope`. There is a key design choice here—why not let it automatically inherit all `todo`? Because "automatic inheritance" equals handing the privilege decision back to the flow default, returning to the owner-mode old path. The meaning of forcing "application" is: **letting "which stages this owner needs to touch" become an explicitly declared, auditable, rejectable action**, not an implicit default.
2. **Automatic tightening (advance)**: revoked once the stage is done. This step is nearly zero cost, because `advance` already changes `todo` $\rightarrow$ `done`, synchronously removing it in `scope` is only a side action of the same state update. Its value is **letting privileges change with facts**: an owner does not permanently retain write privilege for a stage because it "was once responsible."
3. **Out-of-bounds rejection (validate)**: this is where scope really plays defense. When a misled or instruction-injected owner tries to operate stages outside scope, the system does not rely on "it should know not to" but on the mechanical rule `s  $\notin$  scope  $\rightarrow$  scope_violation`. This is the same family as P2's BFT-lite, P3's circuit breaker: **changing "hoping Agents behave" to "structurally it cannot do it"**.

Table 1 centrally contrasts v7.5.0 and v7.6.0 on the privilege dimension.

Table 1 owner mode vs owner+scope mode

Dimension	v7.5.0 (owner)	v7.6.0 (owner+scope)
Responsibility record	single-point responsibility pointer	single-point responsibility pointer (kept)
Privilege record	implicit = can act on all todo	explicit <code>scope</code> = proper subset of todo
Out-of-bounds behavior	no mechanical interception	<code>validate</code> returns <code>scope_violation</code> , no side effects
Privilege after task completion	not automatically revoked	<code>advance</code> automatically removes from scope (tightening)
Privilege on handoff	new owner defaults to inherit all todo	new owner must explicitly apply and pass <code>validate</code> (re-authorization)
Audit	trace records event sequence numbers	additionally records each scope change and each crossing request
With existing contracts	baseline	incrementally compatible, not overturning P10 six fields

Table 1 note: v7.5.0 behavior is P10's verified contract; v7.6.0 behavior is this paper's design proposal, its defense effect to be measured by factorial experiment, [RESULT NEEDED] .

5 Interfaces with UDOS Existing Papers

- **To P10 (TransferBundle):** scope is the seventh field, not changing `goal/context/done/todo/trace` semantics. The migration path is "add field, add three invariants, add a test group," not rewrite. When old tasks lack the scope field, default `scope=todo` (falling back to v7.5.0 behavior), ensuring backward compatibility.
- **To P2 (BFT-lite stopping decisions):** P2 decides "whether this leg should stop," P17's scope decides "what this owner is still allowed to touch before stopping." The two are orthogonal: one a **consensus termination** problem, one a **privilege boundary** problem.
- **To P3 (fault governance/circuit breaker):** `scope_violation` is an earlier signal than "error rate exceeding threshold"—it alarms at the **moment** crossing occurs, rather than waiting for the sliding window to accumulate to threshold. Can serve as a circuit breaker's pre-trigger.
- **To P11 (internal task market):** P11's "who wins, does what, paid how much" is the economic dimension; scope is this winner's "which stages allowed to touch" capability dimension. A winner even if economically qualified can only execute within its scope—**sufficient budget does not equal sufficient privilege**.

6 Drawbacks / Failure Boundaries / Falsifiable Conditions

Critical reading must land on "where this design fails," otherwise it is just another self-consistent slogan.

Its drawbacks and limitations:

1. **scope is a "structural guardrail," not "intelligent judgment."** It can intercept "mechanically unauthorized" but not "within scope, yet doing the task wrong." An owner making wrong decisions within its scope, scope completely cannot perceive. Do not misread least privilege as "universal safety."

2. **Over-tightening harms usability.** If scope is cut too fine, normal cross-stage collaboration is also frequently interrupted by `scope_violation`, the system "unable to run because too cautious." scope granularity itself needs tuning, not tighter-is-better.
3. **Re-authorization relies on the new owner honestly applying.** `new_scope ⊆ todo` is mechanically checkable, but "whether `new_scope` is really just enough, not too large or small" still needs higher-level judgment—scope cannot validate scope quality.
4. **ClawArena's precise numbers cannot be directly moved.** This paper borrows its direction (privilege bottleneck, cost-quality decoupling), its concrete thresholds (e.g. "no LLM exceeds 50% privilege precision") not verbatim hit at abstract level, and its scenario is text + multimodal workspace, not containing distributed faults and Byzantine behavior—it measures "**whether a good Leader can manage the team under normal conditions,**" not "**whether the system crashes when the Leader is misled.**" scope's fault-tolerance value, UDOS must test itself, cannot borrow ClawArena's scenario endorsement.

Falsifiable conditions (design proposals must also be killable):

- **F1:** if in the factorial experiment, owner+scope relative to pure owner has no significant improvement in crossing rejection rate (Wilson CI spans 0), then scope's "automatic tightening" is redundant complexity, should fall back to owner mode.
- **F2:** if scope mode significantly lowers normal task completion rate or latency (e.g. crossing misjudgment also blocking normal collaboration), and this cost is greater than crossing benefits, then "least privilege" in the task scenario is a negative-benefit design.
- **F3:** if the real management bottleneck is found not in privilege granting at all but elsewhere (e.g. task decomposition quality), then this paper taking scope as the first handle is misplaced—precisely the question an EPOB-style "failure distribution rather than only success rate" diagnosis answers.

7 Preregistered Experiment and Honest Boundary

scope's benefits currently are **design proposal • to implement**. This paper fills no "reduce cost X%," "reduce error Y%" numbers. The suggested preregistered factorial experiment framework is:

- **Factors:** privilege mode {owner baseline, owner+scope} × chain length {1,2,3,5} × fault injection {none, unauthorized injection, misled honest owner}.
- **Metrics:** `scope_violation` trigger count, whether crossing writes really rejected (I2 hit rate), normal task completion rate, completion latency, mis-isolation rate.
- **Criteria:** under the premise of normal completion rate not significantly dropping (F2 threshold), whether crossing write rejection rate significantly rises (F1 criterion).
- **Number state:** all result positions [RESULT NEEDED: `scope factorial experiment`], to supplement after the experiment runs, before submission needing ≥ 30 seeds and reporting 95% CI.

8 Conclusion

v7.5.0's TransferBundle solved "no one closes," v7.6.0's owner+scope tries to solve "privilege boundary crossing." Its motivation comes from a directional hint of ClawArena-Team: management bottleneck in privilege granting, cost decoupled from management quality—so the correct lever is "cut privileges smaller, tighten with the flow, re-authorize on handoff," not "change the Leader model bigger."

This paper did not claim this has been measured and proven. Instead, this paper writes scope as three testable invariants, a group of factorial experiments to run, and marks every expected benefit as [RESULT NEEDED] . Structural prior determines the upper limit—a structural choice like owner determines on which failure class the system overturns; while upgrading owner to owner+scope is further changing "hoping Agents behave" to "structurally it cannot cross." This is P17's increment for v7.6.0.

References

External dialogue object (existence verified online 2026-09-21; its precise thresholds are paper reports, not independently reviewed)

1. ClawArena-Team: Benchmarking Subagent Orchestration and Dynamic Workflows in Language-Model Agents. arXiv:2606.31174 (paper direction: management bottleneck concentrating in privilege granting, cost approximately decoupled from management quality, pure execution-style scoring no LLM judge; precise thresholds like "no LLM exceeds 50% privilege precision," "cost hundredfold/score fourfold," "SMS three-factor product formula" not verbatim hit at abstract level, relayed as "paper report • not independently reviewed").

Classic background

1. Saltzer, J. H., Schroeder, M. D. (1975). The Protection of Information in Computer Systems. Proceedings of the IEEE (least privilege/access control classic; this paper only borrowing its "least privilege" principle, not re-arguing).

UDOS paper volumes (this series, evidence grades marked in text)

1. UDOS v7.5.0 paper volume: P2 "BFT-lite safety verification of multi-agent stopping decisions," P3 (fault governance/circuit breaker, v7.5.0 corresponding paper), P10 "Agent-as-Tool and TransferBundle handoff information loss measurability" (six-field contract, 26 tests verified), P11 "Internal task market settlement conservation and Byzantine slashing mechanism design," P12 "Evidence-grading-driven reproducible Agent systems engineering."
2. UDOS v7.6.0 paper volume: P16 "Typed finality," P32 "Rules before learning" (scope_violation belongs to rule predicates, consistent with P32's "rules as base, learning as supplement" orientation).

Evidence Discipline and Reproduction Notes

- This paper is a **design proposal**, no new UDOS measurement; scope's three invariants and factorial experiment are both to-implement designs, all benefit positions [RESULT NEEDED: scope factorial experiment] .
- ClawArena-Team's precise thresholds (privilege precision percentage, cost/quality multiples, SMS formula) were not verbatim hit in this abstract-level verification, this paper does not hardcode any concrete percentage, only citing its directional conclusions; when citing must keep the "paper report • not independently reviewed" caliber.
- v7.5.0 existing contracts (six fields, 26 tests) point back to P10, evidence grade verified; scope is an increment field on it, backward compatible (default scope=todo falls back to owner baseline).
- This paper does not claim "least privilege = universal safety"; scope only intercepts structural crossing, not wrong decisions within scope, also needing to prevent over-tightening harming usability (see § 6 drawbacks and F2 criterion).